Exercise-Scoped Math Practice Generator — Spec

Oct 31, 2026 · @Rabindra

1. Summary and core decision

A student picks a chapter and an exercise from one Algebra textbook, and the app returns 2–3 new practice questions built only from the concepts that exercise teaches. The key decision: **do not train or fine-tune a model.** Use exercise-scoped retrieval, a constrained LLM call, and automatic verification of every generated question.

- **Why not train:** a textbook with 6 chapters and dozens of exercises gives a few examples per exercise. Per-exercise fine-tuning overfits on that little data, costs more, is hard to update, and still hallucinates. Putting only that exercise's material in the prompt gives the same "use only this data" effect.
- **Scoping is a hard filter, not similarity search.** Chapter 3, Exercise 9 maps to an `exercise_id`. The prompt context is built from rows tagged with that ID (its questions, worked examples, concept notes). Nothing from other exercises enters the prompt.
- **Anti-drift stack:** hard-filtered context, strict JSON output schema, SymPy check of the answer, an LLM-judge check that each question uses only the allowed concepts, then retry or fall back to a stored question.
- **v1 scope:** one textbook, no student accounts, 2–3 questions per request, each with a final answer and a step-by-step solution.
- **Out of v1:** logins, handwritten input, grading of student answers, multiple textbooks, progress tracking.

2. Tech stack

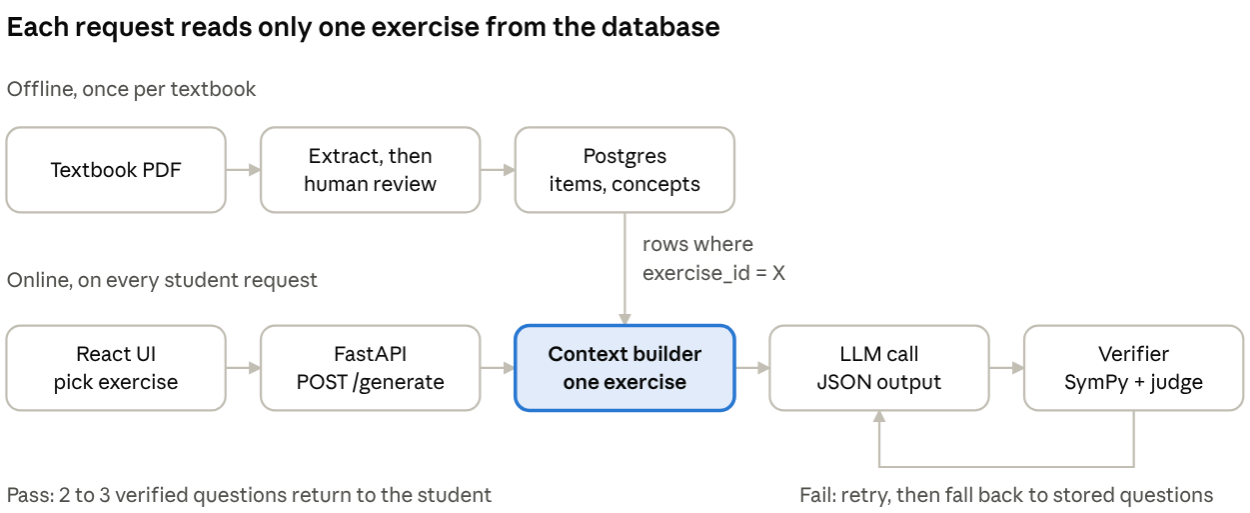
FastAPI and React are the right picks; nothing better is needed for this scope. Skip LangChain and LlamaIndex: the pipeline is a few plain functions, which is easier for Codex to build and for you to debug.

Layer	Choice	Why
Backend	Python 3.12, FastAPI, Pydantic v2, Uvicorn	Typed request and response models, auto OpenAPI docs, async LLM calls

Layer	Choice	Why
Database	PostgreSQL 16 with pgvector; SQLAlchemy 2, Alembic	Relational filter by exercise ID now, vector search later without a migration to a new store
LLM	Anthropic Claude API behind an `LLMClient` interface; generator model set by env var (default `claude-sonnet-5-5`), judge model `claude-haiku-5-5`	Reliable JSON output, cheap judge pass; the interface lets you swap providers
Math checking	SymPy	Verifies final answers and equivalence of expressions without trusting the LLM
Ingestion	PyMuPDF for text, Mathpix or Marker for math-to-LaTeX, then human review of a JSON file	Six chapters is small; reviewed data beats a fully automatic parser
Frontend	React 18, Vite, TypeScript, TanStack Query, Tailwind, KaTeX	Fast setup, server-state caching, fast math rendering
Caching and limits	Generated sets cached in Postgres; `slowapi` rate limit per IP	Controls cost with no extra service in v1
Tests	pytest, Vitest, React Testing Library	Covers backend pipeline and UI states
Deploy	Docker Compose locally; backend on Render or Fly.io, frontend on Vercel	Simple and cheap for a first release

3. Architecture

An offline job fills Postgres once; every online request reads only the rows for the chosen exercise.



ingestion and generation flow · 8 steps, 1 retry loop

Ingestion (offline, once per textbook)

1. Export each chapter to text and LaTeX (PyMuPDF for text; Mathpix or Marker for equations).
2. Split into chapter, exercise and item (questions, worked examples, concept notes) and write `data/textbook.json`.
3. A human reviews the JSON: fixes LaTeX, adds `concepts[]` tags, answers and `difficulty` per item.
4. A seed script validates the JSON with Pydantic and loads it into Postgres. Embeddings are optional in v1.

Generation (per request)

1. The UI sends `exercise_id`, `count` (2–3) and an optional `difficulty`.
2. Cache check: serve verified questions this session has not seen first; generate only when fewer than `count` remain.
3. The context builder loads items where `exercise_id = X`, plus the concept notes those items are tagged with. It never queries other exercises.
4. The LLM is called with the system prompt, the context pack and a JSON schema (section 5).
5. The verifier runs schema validation, a SymPy answer check, a duplicate check against source items, and an LLM judge for concept scope.
6. Pass: save and return. Fail: retry up to 2 times with the failure reasons added to the prompt, then fall back to curated questions from the bank.

4. Data model

Every item hangs off an `exercise_id`, which is what makes the hard filter possible. Use stable slug IDs such as `alg-ch3-ex9`; all tables use UUID primary keys plus a unique `slug` where noted.

Table	Key fields	Notes
`textbooks`	`id`, `slug`, `title`, `edition`, `grade_level`	One row in v1
`chapters`	`id`, `textbook_id`, `number`, `title`	6 rows
`exercises`	`id`, `slug`, `chapter_id`, `number`, `title`, `page_start`, `page_end`	`slug` like `alg-ch3-ex9`
`concepts`	`id`, `slug`, `chapter_id`, `name`, `summary_md`, `formulas_latex[]`	Allowed-concept vocabulary used by the generator and judge
`exercise_concepts`	`exercise_id`, `concept_id`	Many-to-many; defines the scope of each exercise
`items`	`id`, `exercise_id`, `kind`, `body_latex`, `answer_latex`, `solution_steps` (jsonb), `difficulty` (1–3), `question_type`, `source_page`, `embedding` (vector, nullable)	`kind` is `question`, `worked_example` or `concept_note`
`generated_questions`	`id`, `exercise_id`, `prompt_latex`, `answer_latex`, `solution_steps` (jsonb), `difficulty`, `concept_slugs[]`, `status`, `verification` (jsonb), `model`, `prompt_version`, `content_hash`, `created_at`	`status` is `verified` or `rejected`; only `verified` is ever served; unique on `content_hash`
`served_questions`	`session_id`, `generated_question_id`, `served_at`	Anonymous session ID from the browser; stops repeats

Table	Key fields	Notes
`generation_logs`	`id`, `request_id`, `exercise_id`, `attempts`, `outcome`, `failure_reasons` (jsonb), `tokens_in`, `tokens_out`, `latency_ms`, `created_at`	`outcome` is `generated`, `cache`, `fallback` or `error`

Seed file shape (`data/textbook.json`, validated by Pydantic before load):

```
{
  "textbook": {"slug": "algebra-1", "title": "Algebra"},
  "chapters": [{
    "number": 3, "title": "Linear Equations",
    "concepts": [{"slug": "two-step-equations", "name": "Two-step equations",
"summary_md": "...", "formulas_latex": ["ax+b=c"]}],
    "exercises": [{
      "number": 9, "title": "Exercise 3.9",
      "concept_slugs": ["two-step-equations"],
      "items": [{"kind": "question", "body_latex": "3x+5=20", "answer_latex":
"x=5", "difficulty": 1, "question_type": "solve_linear", "solution_steps":
["3x=15", "x=5"]}]
    }]
  }]
}
```

5. Generation and anti-hallucination design

The LLM never sees anything outside the chosen exercise, and no question reaches a student until a program, not the LLM, has confirmed its answer.

Context pack (built by `context_builder.py`, in this order): exercise title and chapter; allowed concepts (name, summary, formulas); up to 6 `question` items; up to 3 `worked_example` items; the exercise's `question_type` values. Total prompt context target: under 3,000 tokens.

System prompt template (stored in `prompts/generate_v1.md`, version recorded with every generated question):

```
You write practice questions for a student working on {chapter_title},
{exercise_title}.
Use ONLY the concepts and techniques in ALLOWED_CONCEPTS. Do not introduce
any other method.
Match the style and question_type of the SAMPLE_QUESTIONS but use different
numbers and, where natural, different wording.
Never copy a sample question. Difficulty: {difficulty_instruction}.
Return exactly {count} questions as JSON matching the schema. Every question
must include a machine-checkable `verify` block in SymPy syntax.
If the context is not enough to write a valid question, return an empty list
and set `refusal_reason`.

ALLOWED_CONCEPTS: {concepts}
SAMPLE_QUESTIONS: {questions}
WORKED_EXAMPLES: {examples}
```

Output schema (Pydantic model `GenerationResult`, sent to the LLM as a JSON schema or tool definition):

```
{
  "questions": [{
    "prompt_latex": "string",
    "answer_latex": "string",
    "solution_steps": ["string"],
    "concept_slugs": ["string"],
    "difficulty": 1,
    "verify": {"type": "solve_equation", "variable": "x", "expression":
"3*x+5=20", "expected": "5"}
  }],
  "refusal_reason": null
}
```

Verification layers (all in `verifier/`, run in this order; the first failure stops the question):

Layer	Check	On failure
1. Schema	Pydantic validates fields; question count equals `count`; `concept_slugs` is a subset of the exercise's allowed slugs	Reject set, retry

Layer	Check	On failure
2. Math	SymPy independently evaluates `verify` (types: `solve_equation`, `simplify_equivalence`, `factor_equivalence`, `evaluate`, `solve_inequality`, `solve_system`) and compares to `answer_latex`	Reject question, retry
3. Scope	LLM judge (temperature 0) reads question plus solution steps and answers JSON: `uses_only_allowed_concepts`, `matches_question_type`, `age_appropriate`	Reject question, retry with judge notes
4. Novelty	Normalized `content_hash` must differ from every source item and from stored generations	Reject question, retry
5. Answerable	Prompt contains no missing data; solution steps end at the stated answer	Reject question, retry

Other rules:

- **Retries:** at most 2 regenerations, each with the failed layer and reason appended to the prompt. Generation temperature 0.7, judge temperature 0.
- **Fallback:** if still short of `count`, serve verified questions from the bank (`generated_questions` with `status = verified`, else source `items`), and flag `source: "bank"` in the response so the UI can say so.
- **Difficulty:** `same` (default) matches the exercise's items; `easier` and `harder` shift by one level using `difficulty_instruction` text in the prompt, never by adding new concepts.
- **No answers in the prompt body:** the UI hides answers and solutions until the student taps Reveal.
- **Logging:** every request writes a `generation_logs` row, including rejected questions and reasons, so prompts can be tuned from real failures.

6. REST API contract

All routes are under `/api/v1`, JSON only, `snake_case` fields. Every response carries an `X-Request-ID` header that matches `generation_logs.request_id`. CORS allows only the origin set in `FRONTEND_ORIGIN`.

Method and path	Purpose	Response
`GET /health`	Liveness and DB check	`{"status": "ok"}`
`GET /chapters`	Chapter picker	`[{id, number, title, exercise_count}]`
`GET /chapters/{chapter_id}/exercises`	Exercise picker	`[{id, slug, number, title}]`
`GET /exercises/{slug}`	Exercise header and concepts shown above the questions	`{slug, number, title, chapter, concepts: [{slug, name}]}`
`POST /exercises/{slug}/generate`	Generate practice questions	See below

Generate request

```
{ "count": 3, "difficulty": "same", "session_id": "b1c2...uuid" }
```

`count` is 2 or 3 (default 3); `difficulty` is `same`, `easier` or `harder` (default `same`); `session_id` is a UUID the browser creates and stores in `localStorage`.

Generate response (200)

```
{
  "exercise_slug": "alg-ch3-ex9",
  "source": "generated",
  "questions": [{
    "id": "uuid",
    "prompt_latex": "4x-7=21",
    "answer_latex": "x=7",
    "solution_steps": ["4x=28", "x=7"],
    "difficulty": 1,
    "concept_slugs": ["two-step-equations"]
  }]
}
```

`source` is `generated`, `cache` or `bank`. The UI shows a small notice for `bank`.

Errors use `{"error": {"code": "...", "message": "..."}}`:

Status	Code	When
404	`exercise_not_found`	Unknown slug
422	`invalid_request`	Bad `count`, `difficulty` or `session_id`
429	`rate_limited`	Over the per-IP limit (default 20 generate calls per minute)
503	`generation_unavailable`	LLM down and the bank has fewer than `count` questions

Seeding is a CLI command (`python -m app.cli seed data/textbook.json`), not an API route, in v1.

7. Frontend spec

A single-page React app with one screen and three steps: choose chapter, choose exercise, practice. It works on a phone first, since students will mostly use one.

Screen flow

1. **Chapter picker:** cards or a select listing the 6 chapters from `GET /chapters`.
2. **Exercise picker:** appears after a chapter is chosen; lists its exercises from `GET /chapters/{id}/exercises`. Changing the chapter clears the exercise.
3. **Practice panel:** after an exercise is chosen, show the exercise title and its concept chips, then a **Generate practice questions** button. Clicking it calls `POST /exercises/{slug}/generate`.

Practice panel behavior

- Each question renders as a card: number, KaTeX-rendered prompt, a **Show hint** toggle (first solution step), a **Reveal answer** toggle (final answer), and a **Show full solution** toggle (all steps). All three are hidden by default.
- Controls: difficulty select (Same, Easier, Harder) and a **Generate more** button; each new batch replaces the old one and the old batch stays reachable through a Back control for the current visit.
- States to implement: idle, loading (skeleton cards, button disabled, typical wait 3–10 seconds), success, bank notice when `source = bank` ("Showing practice questions from our saved set"), error with Retry, rate-limited (disable button with the wait time), empty exercise list.
- Selected chapter and exercise are kept in the URL (`/?chapter=3&exercise=alg-ch3-ex9`) so a student can bookmark or share the exact practice page.

Components

Component	Responsibility
`ChapterSelect`, `ExerciseSelect`	Pickers; fetch with TanStack Query; keyboard accessible
`ConceptChips`	Shows the concepts the exercise covers
`GenerateControls`	Count (2 or 3), difficulty, generate button
`QuestionCard`	One question with hint, answer and solution toggles
`MathText`	Wraps KaTeX; falls back to raw text if rendering throws
`ErrorBanner`, `SkeletonCard`	Shared status UI

Technical rules: TypeScript strict mode; API types generated from the FastAPI OpenAPI schema (`openapi-typescript`); API base URL from `VITE_API_BASE_URL`; a `session_id` UUID created once and kept in `localStorage`; WCAG AA contrast, visible focus states, and all toggles reachable by keyboard.

8. Repo structure, config and Codex conventions

One monorepo with `backend/` and `frontend/` so Codex can see the API contract and the UI together.

```
math-practice/
  AGENTS.md                 # rules for Codex (below)
  docker-compose.yml        # postgres + backend + frontend
  data/textbook.json        # reviewed seed data (not committed if
copyrighted)
  backend/
    pyproject.toml
    alembic/
    app/
      main.py               # FastAPI app, CORS, routers
      config.py             # pydantic-settings
      cli.py                # seed, export, eval commands
      db/                   # models.py, session.py
      api/                  # chapters.py, exercises.py, generate.py
      schemas/              # request, response and GenerationResult models
      services/
        context_builder.py
```

```
        generator.py        # orchestrates retry and fallback
        llm_client.py       # LLMClient interface + Anthropic implementation
        cache.py
      verifier/             # schema.py, math_check.py, judge.py, novelty.py
      prompts/              # generate_v1.md, judge_v1.md
      ingest/               # extract.py, validate.py, seed.py
    tests/
  frontend/
    package.json
    src/ (components/, api/, hooks/, pages/)
```

Environment variables

Variable	Used by	Example
`DATABASE_URL`	backend	`postgresql+psycopg://user:pass@db:5432/math practice`
`ANTHROPIC_API_KEY`	backend	secret, never committed
`GENERATOR_MODEL`	backend	`claude-sonnet-5-5`
`JUDGE_MODEL`	backend	`claude-haiku-5-5`
`MAX_RETRIES`	backend	`2`
`RATE_LIMIT_GENERATE`	backend	`20/minute`
`FRONTEND_ORIGIN`	backend	`http://localhost:5173`
`VITE_API_BASE_URL`	frontend	`http://localhost:8000/api/v1`

`AGENTS.md` rules for Codex (copy into the repo root):

- Build milestones in the order of section 10; finish and test one before starting the next.
- Do not add features, endpoints or tables that this spec does not list; ask instead.
- The LLM is called only through `LLMClient`; tests use a `FakeLLMClient` and never hit the network.
- Every generated question must pass `verifier/` before it is saved or returned; never bypass it.
- The context builder must filter by `exercise_id` in SQL; add a test that proves other exercises' rows never appear.
- Python: type hints everywhere, `ruff` and `mypy` clean. TypeScript: strict mode, `eslint` clean.

- Run `pytest` and `npm test` before declaring a milestone done, and list which acceptance criteria passed.

9. Testing and evaluation

Automated tests protect the code; a separate evaluation run protects question quality, because prompts and models will change.

Area	Tests
Context builder	Chapter 3 Exercise 9 context contains only rows with that `exercise_id`; token budget respected; unknown slug raises
Math verifier	Table of known-good and known-bad `verify` blocks for each type; wrong answer rejected; malformed SymPy rejected without crashing
Generator orchestration	With `FakeLLMClient`: success on first try, success after retry, fallback to bank, all-fail returns 503
Novelty	Copy of a source item and a number-for-number duplicate are both rejected
API	Each route returns the documented shape and error codes; rate limit returns 429
Ingestion	Invalid `textbook.json` (missing answer, unknown concept slug) fails with a clear message; valid file loads idempotently
Frontend	Picker dependency (exercise resets on chapter change), loading, error, bank notice, reveal toggles, KaTeX fallback

Evaluation harness (`python -m app.cli eval`): for every exercise, generate 10 sets with the real model and report:

Metric	Target for v1
Math-check pass rate on first attempt	at least 90%
Questions served after at most 2 retries	at least 99%
Judge says scope is respected	100% of served questions

Metric	Target for v1
Fallback rate	under 5%
Median generation latency	under 8 seconds

Before launch, a teacher or you should hand-review at least 5 generated questions per exercise. The judge catches many problems but is not a substitute for a human checking that questions feel like the textbook.

10. Milestones for Codex

Each milestone is sized for one Codex task or pull request. Give Codex this whole document plus one milestone at a time (for example: "Implement M4 from the spec").

#	Build	Acceptance criteria
M0	Monorepo scaffold, `docker-compose.yml`, FastAPI app with `/health`, Vite React app, lint and test commands, `AGENTS.md`	`docker compose up` serves `/api/v1/health` and the frontend; `ruff`, `mypy`, `eslint`, `pytest`, `npm test` run clean on the empty projects
M1	SQLAlchemy models and first Alembic migration for every table in section 4	Migration applies to an empty Postgres and rolls back; a model test inserts one row per table
M2	Pydantic seed schema, `ingest/validate.py`, `ingest/seed.py`, CLI `seed`, and a sample `data/textbook.json` with 1 chapter and 2 exercises	Seeding twice creates no duplicates; a file with a missing answer or unknown concept slug fails with a readable error
M3	Read endpoints: `/chapters`, `/chapters/{id}/exercises`, `/exercises/{slug}`	Responses match section 6 exactly; 404 on unknown slug; tests pass
M4	`context_builder.py`	Test proves Chapter 3 Exercise 9 context never contains another exercise's rows; output stays under 3,000 tokens on the sample data

#	Build	Acceptance criteria
M5	`LLMClient` interface, Anthropic implementation, `FakeLLMClient`, `prompts/generate_v1.md`, `GenerationResult` model, basic generator	With the fake client the generator returns valid `GenerationResult`; the real client is called only when `ANTHROPIC_API_KEY` is set and is never used in tests
M6	Verifier layers 1–5, retry loop, bank fallback, cache, `served_questions`, `generation_logs`, and `POST /exercises/{slug}/generate`	All cases in section 9 for verifier, orchestration, novelty and API pass; a wrong-answer question is never returned or saved as `verified`
M7	React UI: pickers, practice panel, all states in section 7, KaTeX, URL state, generated API types	Component tests pass; at 375 px width there is no horizontal scroll; keyboard-only use completes the whole flow
M8	Eval CLI, rate limiting, production Dockerfiles, `README.md`, deploy notes	`eval` prints the metrics in section 9 for the sample data; `README` lets a new developer run everything in under 10 minutes

After M8: load the real textbook, run `eval`, tune the prompt, hand-review questions, then release to a small group of students.

11. Risks, copyright and open questions

The biggest non-technical risk is rights to the textbook; the biggest technical one is word problems that SymPy cannot check.

Risk	Mitigation
Textbook copyright: storing its text in a database and showing derived questions on a public site may need the publisher's permission (this is not legal advice; check with the publisher or a lawyer)	Get written permission, or build and test on an openly licensed Algebra book such as OpenStax; keep `data/textbook.json` out of public repos
Word problems have no simple SymPy check	Require the LLM to emit the underlying equation in `verify`; if it cannot, mark the type `judge_only` and require two judge passes; start v1 with equation-style questions
PDF extraction garbles LaTeX	Human review step in ingestion; never seed unreviewed data

Risk	Mitigation
Cost or abuse of the generate endpoint	Cache, per-IP rate limit, 3-question cap, spend alert on the API key
Questions drift in style from the textbook	Include 3 or more source items in every context pack; track the judge's `matches_question_type` rate in `eval`
Latency feels slow	Skeleton UI, cache seen questions, optional background pre-generation of a bank per exercise

Open questions to settle before M2:

- Which textbook and edition, and do you have permission to use its content?
- What grade level, and does the book use special notation or a specific curriculum?
- Should students see the full worked solution, or only hints and the final answer?
- Will you want accounts, progress history or teacher views later? If yes, the `session_id` design changes in v2.
- What monthly budget should the LLM spend stay within?